

Ulsan-Fault earthquake catalog pipeline

Reference manual: detection → association → location → relocation (HypoDD dt.ct + dt.cc)

Long-term (2010–present) catalog for the Ulsan Fault, SE Korea

src/ufpipe + detection_test

`ufpipe` (`src/ufpipe`) is the **end-to-end pipeline**: six stages, detection → association → augment → PHS → locate → **relocate** (HypoDD dt.ct + dt.cc). The final **relocate** stage produces the double-difference relocated catalog. The heavy dt.cc/xcorr/HypoDD *engine* is external (`15.PocketQuake` korea-cluster-relocation, HypoDD v2.1beta); `ufpipe` orchestrates it. The relocate stage is **self-fed**: it builds its inputs from `ufpipe`’s OWN detection+association (`ufpipe.reloc_inputs`), so `detection_test/lib/` is DEPRECATED (see §7 + `detection_test/lib/DEPRECATED.md`); only the downstream reloc driver + PocketQuake engine remain live. The relocation internals have a dedicated guide: `detection_test/reloc_2016_uf/PIPELINE.md` + `study_guide.pdf`.

Contents

1	What the pipeline does	1
1.1	<code>ufpipe</code> vs <code>detection_test</code>	2
1.2	Two orthogonal “model” dimensions	2
2	Install & environment	3
3	Stages 1–5: detection through location	3
3.1	Source of truth for pick↔event assignment	4
4	Parameters (<code>src/ufpipe/config.py</code> — the single disclosed source)	4
4.1	Detection / DSP	4
4.2	Association (PyOcto) — daily-chunked, ASSOC_GATE (lenient) vs ASSOC_GATE_STRICT .	4
4.3	Velocity model & location	5
5	How to run	5
5.1	Running each stage separately	6
5.2	Relocation — running the last stage (stage 6)	6
6	Outputs & layout	7
7	Stage 6: relocation — HypoDD dt.ct + dt.cc	7
8	Robustness & invariants	8
9	Troubleshooting / provenance checks	9

1 What the pipeline does

Build an **internally consistent, long-term earthquake catalog** for the Ulsan Fault region (SE Korea), 2010–present, from continuous seismic waveforms, using AI phase pickers so small events are detected uniformly even as the recording network grows, and finally **relocate** the catalog with

double-difference (HypoDD dt.ct + dt.cc) for sub-100 m relative precision. Every stage is scripted and parameterized; results reproducible.

Detection	Association	Location	QC	Relocation
PhaseNet / PhaseNet+	PyOcto (+augment)	HYPONVERSE	uf_cluster	HypoDD dt.ct + dt.cc
<i>picks</i> →	<i>events+assign</i> →	<i>.sum/.arc</i> →	<i>well-located</i> →	<i>hypoDD.reloc</i>

1.1 ufpipes vs detection_test

ufpipe is the production end-to-end pipeline. `detection_test` is the pilot experiment (the 2016 4-picker comparison). Its `lib/` (per-month detection/association) is DEPRECATED — ufpipes feeds itself now — and only the relocation `orchestrator reloc_2016_uf/run_picker_reloc.py` (driven by ufpipes’s relocate stage with `--skip-build`) + the external HypoDD engine remain live there.

	ufpipes (src/ufpipes)	detection_test
Role	the self-contained end-to-end catalog + relocation pipeline	4-picker-comparison pilot; hosts the reloc driver + PocketQuake engine (<code>lib/</code> DEPRECATED)
Stages	detection → association → augment → PHS → locate → relocate	reloc driver (<code>reloc_2016_uf/</code> , <code>--skip-build</code>)
Driver	<code>python -m ufpipes.run_pipeline</code>	(called by ufpipes’s relocate stage)
Ends at	HypoDD dt.cc (§7)	—

ufpipes’s own detection + association cover **KS/KG/GJ/NS** with daily-chunked association (§5), and the `relocate` stage builds its inputs from those outputs (`ufpipes.reloc_inputs`); it no longer consumes `detection_test/lib`. The pipeline is fully self-contained. See §7.

1.2 Two orthogonal “model” dimensions

Dimension	Flag	Values
Picker model	<code>--model</code>	<code>stead</code> , <code>original</code> , <code>phasenet_plus</code> , <code>eqt</code> , ...
Velocity model	<code>--velmodel</code>	<code>kim1983</code> , <code>kim2011</code>

Any picker model can be located with any velocity model. `stead/original` are SeisBench PhaseNet weights; `phasenet_plus` is EQNet PhaseNet+ (needs a local EQNet clone).

2 Install & environment

One-time setup

1. `conda env create -f environment.yml && conda activate ulsan` (the code also runs in base here)
2. `pip install torch --index-url https://download.pytorch.org/whl/cu128` (match your CUDA; GPU strongly recommended)
3. `pip install -e .` (installs the `uflib` + `ufpipe` packages, editable)
4. **HYPOINVERSE**: the external `hyp1.40` binary must be on `PATH` (not pip-installable).
5. **PhaseNet+** only: a local EQNet clone at `config.EQNET_DIR`.

After `pip install -e .`, `import ufpipes.config` and `python -m ufpipes.run_pipeline` work from any directory — no `sys.path` hacks.

3 Stages 1–5: detection through location

`STAGES = [detection, association, augment, phs, locate, relocate]` — run in order. Stages 1–5 (below) are functions in `src/ufpipe/core.py`; the relocate stage (6) is `src/ufpipe/relocate.py` and has its own §7. `run_pipeline.py` chains them per year.

#	Stage	In → Out	What it does
1	detection	continuous mSEED → picks/ (per station-day)	AI picker on each station-day across KS/KG/GJ/NS (per-year multi-network table, <code>stations.py</code>); anti-alias >100 Hz, unify mixed rates, resample to 100 Hz, pick P/S above threshold. GPU. Idempotent (skips done days).
2	association	picks → <code>pyocto_*.csv</code> + assignment	PyOcto octree associator groups picks into events (gate below), daily-chunked (± 150 s/day, in-day origins) so it scales to the dense NS array. Coords from the multi-network table.
3	augment	assignment → augmented assignment	scan daily picks for orphans PyOcto missed, add to each event so HYPOINVERSE sees the full pick set. Backs up + overwrites in place; idempotent.
4	phs	assignment → <code>UF{year}.phs</code>	write the HYPOINVERSE phase file from the (augmented) assignment.

#	Stage	In → Out	What it does
5	locate	<code>.phs → UF{year}.{sum,arc}</code>	run HYPOINVERSE (hyp1.40) with the chosen velocity model → located catalog + errors.

Pipeline order matters

augment runs *between* **association** and **phs**, so HYPOINVERSE sees the full augmented pick set and computes correct GAP/DMIN/ERZ/RMS. Do not reorder.

3.1 Source of truth for pick↔event assignment

The **PyOcto assignment (after augmentation)** is authoritative for which (station, phase) tuples belong to each event. Do NOT use a legacy time-window pick dump — when two events fall <60 s apart at the same station the earliest pick “wins” and leaks into the wrong event. Downstream (SAC-header writers, magnitude/QC notebooks) must read `pyocto_assignment_{velmodel}_{year}.csv`.

4 Parameters (src/ufpipe/config.py — the single disclosed source)

Change defaults here. Everything is one file.

4.1 Detection / DSP

Parameter	Value	Meaning
SAMPLING_RATE	100.0 Hz	the pickers’ trained rate; all data resampled to it
ANTIALIAS	lowpass 45 Hz, 8 corners, zerophase	applied BEFORE down-sampling (>100 Hz KG/NS)
BANDPASS	1–40 Hz, 4 corners, causal	legacy detection path; PhaseNet+ runs raw
P_THRESHOLD, S_THRESHOLD	0.2	SeisBench pick probability threshold
PNPLUS_MIN_PROB	0.3	EQNet PhaseNet+ pick threshold
PNPLUS_NT	1024×36	PhaseNet+ patch length (whole-day, cut_patch)
MAX_CORES	24	CPU cap (respects taskset affinity)

4.2 Association (PyOcto) — daily-chunked, ASSOC_GATE (lenient) vs ASSOC_GATE_STRICT

Association runs **one calendar day at a time**: a $\pm$ ASSOC_OVERLAP_S (150 s) window per day is associated and only events whose *origin* falls in-day are kept (dedup). This is physically equivalent to a single pass (local events are seconds long) but keeps each PyOcto solve to seconds — a whole-year single pass is intractable on the dense $\sim$ 200-station NS array ($\gg$ 1 h, 12 GB). Station coordinates come from the multi-network year table (`stations.py`), so KS/KG/GJ/NS all associate.

Parameter (config)	lenient	_STRICT	Meaning
REGION_CENTER $\pm$ pad	(35.856,129.224) $\pm$ (1.0,1.2)		search area (deg)
ASSOC_ZLIM	(0, 30)		depth range (km)
ASSOC_TIME_BEFORE	300		origin-search window (s)
ASSOC_OVERLAP_S	150		daily-chunk overlap (s)
n_picks	4	6	min total picks
n_p / n_s	2 / 2	3 / 3	min P / S picks
n_ps	1	2	min stations with both P+S (depth)
ASSOC_PICK_MATCH_TOL		1.5	residual cap (s)

Pass `--strict` to use `ASSOC_GATE_STRICT` (fewer, cleaner events). `--workers N` parallelises the independent daily chunks. Velocity for association = `ASSOC_VELMODEL` (`kim2011` 1-D), matching the reloc feeder. Output schema (events `idx,time,x,y,z,picks,latitude,longitude,depth`; assignment `event_idx,pick_idx,residual,station,phase,time`) is unchanged, so `phs/locate/relocate` consume it as before.

4.3 Velocity model & location

`DEFAULT_VELMODEL` = `kim2011`; `kim1983` also available (`data/metadata/velocity_model/kim1983.csv`, a layered V_p/V_s model). `PHASE_CHANNELS` = {P: HHZ, S: HHN}.

5 How to run

Standard runs (from the repo root)

```
python -m ufpipes.run_pipeline --model original --years 2024          one year, all 5 stages
python -m ufpipes.run_pipeline --model original --years 2010-2024    the whole record
python -m ufpipes.run_pipeline --model original --years 2015,2016 --velmodel kim1983    specific years, kim1983
python -m ufpipes.run_pipeline --model original --years 2024 --stage-from association    resume (picks already done)
python -m ufpipes.run_pipeline --model original --years 2024 --days 1-3    detection on a few days (testing)
python -m ufpipes.run_pipeline --model original --years 2024 --strict        strict association gate
python -m ufpipes.run_pipeline --model original --years 2016 --stage-from relocate --through dtcc    relocation only (dt.cc; needs feeder)
```

CLI flags: `--model` · `--years` ('2010-2024' or '2015,2016') · `--velmodel` · `--days` · `--device` {cuda,cpu} · `--workers` · `--strict` · `--force` (allow writing to stead) · `--stage-from` {detection,association,augment,phs,locate} (start mid-chain).

Shared 64-core server — always pin cores

Detection sizes its preprocessing pool and `torch` threads from the process CPU *affinity*, capped by `MAX_CORES=24`. Launch under `taskset` so the whole job stays within budget:

```
OMP_NUM_THREADS=1 taskset -c 0-7 python -m ufpipes.run_pipeline --model original
--years 2010-2024
```

Without pinning, PhaseNet+ grabbed ~49 cores / 193 threads and starved everything. Inference is **GPU-preferred** (warns loudly, never silently falls back to CPU).

Detection is **idempotent** — it skips station-days whose picks already exist, so runs resume safely after an interruption.

5.1 Running each stage separately

Every `ufpipe` stage is also a standalone CLI (`--model --year`), so you can run detection alone, then association alone, etc. Order is `detection` → `association` → `augment` → `phs` → `locate`. Each reads the previous stage's output from disk.

Stage	standalone command
Detection	<code>python -m ufpipes.detection --model original --year 2024</code> <code>[--networks KS,KG,GJ,NS] [--days 1-5] [--workers 8]</code>
Association	<code>python -m ufpipes.association --model original --year 2024</code> <code>[--networks KS,KG,GJ,NS] [--workers 8] [--strict]</code>
Augment	(runs inside <code>run_pipeline</code> ; or call <code>core.run_augment_year</code> from Python)
PHS	<code>python -m ufpipes.make_phs --model original --year 2024</code>
Location	<code>python -m ufpipes.run_hypoinverse --model original --year 2024</code> <code>--velmodel kim2011</code>

Or drive the same stage functions from Python / a notebook:

```
from ufpipes import core, config
core.run_detection_year("original", 2024, days=range(1,6))
events, assign = core.run_association_year("original", 2024)
core.write_phs("original", 2024); core.run_hypoinverse_year("original", 2024,
velmodel="kim2011")
```

5.2 Relocation — running the last stage (stage 6)

The relocate stage is **self-fed**: it needs only `ufpipe`'s own per-year detection + association (`pyocto_kim2011_<year>.csv` + assignment). It builds the reloc input store (event-idx SAC + station table) from those via `ufpipes.reloc_inputs`, then hands off to the driver `run_picker_reloc.py` `--skip-build` (scaffold → HYPOINVERSE → QC → rereference → GPU xcorr → HypoDD dt.cc). If a year's association is missing, the stage preflights and prints exactly what to run.

Step	command
Detection	<code>conda run -n eqnet python -m ufpipes.detection --model original --year 2016</code>
Association	<code>conda run -n base python -m ufpipes.association --model original --year 2016</code>
Reloc (to QC)	<code>python -m ufpipes.run_pipeline --model original --years 2016 --stage-from relocate --through hypoinverse (fast)</code>
Reloc (full)	<code>python -m ufpipes.run_pipeline --model original --years 2016 --stage-from relocate --through dtcc (+ xcorr→HypoDD dt.cc; ~6 h dense)</code>

`--through hypoinverse` stops at the QC'd absolute-location subset; `--through dtcc` continues to the double-difference relocation. `--clean-cache` deletes the ~tens-of-GB interp cache after `dt.cc` (essential for multi-year runs). `detection_test/lib/` is DEPRECATED (see `lib/DEPRECATED.md`); the relocation internals are documented in `reloc_2016_uf/PIPELINE.md` + `study_guide.pdf`.

6 Outputs & layout

All pipeline products go under `outputs/models/<model>/` (regenerable; gitignored):

Path	Contents
<code>.../detection_location/<year>/picks/</code>	per-station-day picks (stage 1)
<code>.../pyocto/pyocto_{vm}_{year}.csv</code>	associated events (stage 2)
<code>.../pyocto/pyocto_assignment_{vm}_{year}.csv</code>	pick→event assignment (authoritative)
<code>.../station_table/stations_{year}.csv</code>	per-year station roster (discovered from metadata)
<code>.../HypoInv/PHS/UF{year}.phs</code>	HYPONVERSE phase file (stage 4)
<code>.../HypoInv/{vm}/UF{year}.{sum,arc,prt}</code>	located catalog + errors (stage 5)

Inputs (restructured 2026-07): waveforms in `KS_KG/` (station dirs at root, parallel to `GJ/,NS/`); station metadata in `data/metadata/ks_kg_station_table/`; velocity model in `data/metadata/velocity_model/`; HYPONVERSE control inputs (`STA/kim*`) in `data/hypoinv/`.

7 Stage 6: relocation — HypoDD dt.ct + dt.cc

The 6th ufpipes stage: turn the QC'd absolute-location catalog into a **double-difference relocated** catalog for sub-100 m relative precision.

```
python -m ufpipes.run_pipeline --model original --years 2016 --stage-from relocate
--through dtcc --clean-cache
```

`ufpipes.relocate` (`src/ufpipes/relocate.py`) is a *thin wrapper*: it pre-flights the per-month feeder inputs then invokes the validated orchestrator `detection_test/reloc_2016_uf/run_picker_reloc.py`, which drives the external HypoDD/xcorr engine. Internal stages (after the per-month detection + association feeder, `KS/KG/GJ/NS`):

#	Stage	What it does
1	SAC store	event-idx-keyed native-rate SAC waveforms from the associated picks (<code>event_sac/</code>)
2	HYPOINVERSE	absolute location of all UF-box events (kim2011) → <code>.sum</code>
3	QC	<code>uf_cluster.QC</code> : keep <code>erh<5</code> , <code>erz<5</code> , <code>gap<270</code> , <code>num>5</code> , <code>rms<1.0</code>
4	re-reference	stamp each SAC's origin from the (full-run) HYPOINVERSE <code>.sum</code>
5	ph2dt	build <code>event.dat</code> + catalog differential times <code>dt.ct</code>
6	xcorr (GPU)	cross-correlation differential times <code>dt.cc</code> (band 5–20 Hz, interp 1000 Hz, <code>cc≥0.7</code>)
7	HypoDD	kim2011, ISTART=2, ISOLV=2 (LSQR), adaptive damping (CND 40–80), HypoDD v2.1beta (MAXDATA=15M) → <code>hypoDD.reloc</code>

Relocation outputs (symlinked into the working dir)

Each run publishes `detection_test/reloc_<year>_uf[_<p>]/results/` with symlinks to the external run tree: `hypoDD.reloc.dtcc` (`dt.ct+dt.cc`), `hypoDD.reloc.dtct` (`dt.ct`-only), `HypoInv.full.sum`, ...“`dt.cc-resolved`” = events keeping ≥ 1 surviving cross-correlation link — the highest-precision subset.

The invariant that matters (see PIPELINE.md)

Origins flow into `dt.cc`: `rereference` stamps SAC origins from a `.sum`, and `xcorr` stores $dt = (t_1 + \text{shift} - ot_1) - (t_2 - ot_2)$ — subtracting those origins. Use the *full-run* HYPOINVERSE solution QC gated on (never a redundant re-run), or the `dt.cc` values are corrupted. This is enforced by `run_picker_reloc.py::inject_full_hypoinverse`.

8 Robustness & invariants

- **Network growth is automatic.** Stations are *discovered from StationXML + on-disk coverage* each month; the deployed network for that period is what enters the run. No hard-coded station lists.
- **Mixed sampling rates handled.** >100 Hz stations (KG/NS at 200 Hz) are anti-alias-lowpassed then resampled to the pickers' 100 Hz. The anti-alias filter is applied BEFORE down-sampling (never after).
- **Velocity model, thresholds, and gates are epoch-invariant by choice** $\Rightarrow$ the catalog stays internally comparable across years.
- **Idempotent & resumable** at every stage; `--stage-from` restarts mid-chain.
- **Do not edit the stead reference run** (it is the baseline). Scripts refuse `--model stead` writes unless `--force`.

9 Troubleshooting / provenance checks

Symptom	Check
Detection finds no waveforms	<code>python -c "import ufpipes.config as c,os; print(c.CONTINUOUS, os.path.isdir(c.CONTINUOUS))"</code> — must point at <code>KS_KG/</code> and exist.
<code>ImportError: ufpipes / uflib</code>	<code>pip install -e .</code> in the active env; <code>import ufpipes.config; print(__file__)</code> must be <code>src/ufpipes</code> .
Wrong pipeline imported	<code>ufpipes</code> is renamed precisely to avoid the collision with the external <code>korea-cluster-relocation pipeline</code> . Verify <code>ufpipes.config.__file__</code> is under <code>src/</code> .
Events look over/under-associated	compare <code>REGION</code> vs <code>--strict</code> ; the <code>pick_match_tolerance</code> is the primary knob.
GPU not used	inference is GPU-preferred and warns loudly; check the torch/CUDA install (never silently CPU).
Sparse early years, few events	expected — <code>KS/KG-only</code> network (2010–2016) has larger azimuthal gaps; the QC gate honestly reflects what the network resolves.

Companion docs: `docs/overview.md`, `docs/pipeline.md`, `docs/how-to-run.md`,
`docs/directory-structure.md` · `src/ufpipes/README.md` · `relocation pipeline:`
`detection_test/reloc_2016_uf/PIPELINE.md` + `study_guide.pdf`.